Laboratorio de Programación (LPR) — 5° Año 3° Grupos A-B

E.E.S.T. N° 1 "Eduardo Ader" — Vicente López

Manual del Programador — Juego de Naves en Consola

v1.0.0 — 21/10/2026

Estudiante: Teo Laguna

Curso / Grupo: 5° Año 3° Grupos A / B

Docente: Mansilla Muñoz York Elías y Yamil Ganduglia

Archivo: src/main.cpp

Estándar: C++11 · Modalidad individual

Paradigma: Programación Orientada a Objetos

Editor: Visual Studio Code (VS Code) con terminal integrada

1. Arquitectura

Un único archivo src/main.cpp. El programa se apoya en tres clases (Nave, Asteroide y Bala) y en dos funciones de bajo nivel para el control de la consola (gotoxy y ocultarCursor). El flujo general es: main() inicializa el juego y contiene el bucle principal; cada clase resuelve su propio dibujo y movimiento.

2. Estructura de datos (clases)

Nave: atributos privados int x, y; e int vidas;. Constructor Nave(int _x, int _y, int _vidas). Métodos: getX(), getY(), getVidas(), reducirVida(), pintar(), borrar() y mover(char).

Asteroide: atributos privados int x, y;, constructor Asteroide(int _x, int _y) y métodos getX(), getY(), pintar(), borrar() y mover().

Bala: atributos privados int x, y;, constructor Bala(int _x, int _y) y métodos getX(), getY(), mover() e fueraDeRango().

3. Flujo detallado

Funciones de consola: gotoxy(int, int) posiciona el cursor con SetConsoleCursorPosition; ocultarCursor() oculta el cursor con SetConsoleCursorInfo.

main(): se siembra el generador con srand(time(NULL)), se pide el nombre del piloto con getline(cin, nombrePiloto), se limpia la pantalla (system("cls")) y se crean el jugador y los vectores de asteroides y balas. El bucle while (!gameOver) actualiza el HUD, lee el teclado y resuelve colisiones, pausando con Sleep(30).

Bucle principal: HUD (PILOTO, VIDAS, SCORE) · lectura con _kbhit() y tolower(_getch()) · movimiento y descarte de balas con fueraDeRango() · movimiento de asteroides y colisiones asteroide-jugador y bala-asteroide.

4. Librerías

- <iostream>: cout, cin, getline y endl.
- <windows.h>: gotoxy, ocultarCursor, Sleep, system("cls") y system("pause").
- <conio.h>: _kbhit y _getch para la lectura del teclado.
- <vector>: contenedores dinámicos vector<Asteroide*> y vector<Bala*>.
- <ctime>: time(NULL) como semilla de aleatoriedad.
- <cctype>: tolower() para normalizar las teclas.

5. Funcionalidad 1 (Score) — disparo doble

La funcionalidad 1 (Score) agrega un contador de puntos y una mejora de disparo:

- int score = 0; en main() y su visualización en el HUD (SCORE).
- score += 10; en el bloque de colisión bala-asteroide (cada asteroide destruido).
- En el if (tecla == ' '): si score < 300 se dispara una bala central; si score >= 300 se disparan dos balas desde las alas (jugador.getX() y jugador.getX() + 4).
- El puntaje final se muestra en Game Over como SCORE FINAL.

```
if (tecla == ' ') {
    if (score < 300) {
        balas.push_back(new Bala(jugador.getX() + 2, jugador.getY() - 1));
    } else {
        balas.push_back(new Bala(jugador.getX(), jugador.getY()));
        balas.push_back(new Bala(jugador.getX() + 4, jugador.getY()));
    }
}
```

6. Puntos críticos

Los vectores almacenan punteros: se usa new al crear y delete antes de reemplazar o eliminar. Al borrar un elemento durante la iteración se decrementa el índice (i--) para no saltearse el siguiente. Los asteroides se recolocan arriba al salir de pantalla, manteniendo la dificultad constante.

7. Ejecución en VS Code

1. Abrir la carpeta juegodenaves/ en Visual Studio Code (File > Open Folder).
2. Abrir la terminal integrada con Ctrl+Ñ (o View > Terminal).
3. Compilar y ejecutar en la terminal:

```
g++ -std=c++11 src/main.cpp -o src/juegodenaves.exe
.\src\juegodenaves.exe
```

También es posible usar Run > Run Without Debugging (Ctrl+F5) si se configura la tarea de compilación, pero la vía recomendada es la terminal integrada.

8. Compilación y pruebas

Pruebas realizadas: carga del nombre con espacios, movimiento con W/A/S/D, disparo con ESPACIO (simple y doble), colisión asteroide-jugador (descuenta vidas), colisión bala-asteroide (suma score) y Game Over al llegar a 0 vidas.

9. Limitaciones y mejoras

- El área de juego está fija (ancho 70, alto 23) y se podría parametrizar.
- No hay límite de disparos simultáneos ni enfriamiento (cooldown).
- El score es un entero simple; podría persistirse en un archivo de récords.
- Agregable: niveles de dificultad y power-ups de vida.

10. Mantenimiento

Los ejecutables (*.exe) permanecen excluidos en .gitignore. Todo cambio de comportamiento que modifique la versión debe registrarse en docs/CHANGELOG.md y actualizar los manuales.